

面向切面编程（AOP）实验指导书

1. 实验基本信息

课程名称：高级程序设计与系统开发工具

实验章节：第13章 面向切面编程（AOP）

前置知识**：Java基础、Spring IoC、反射与注解

2. 实验目的与要求

2.1 实验目的

- 理解AOP核心概念**：掌握切面、连接点、切点、通知等AOP基本概念
- 掌握Spring AOP实现机制**：理解动态代理原理，掌握JDK代理和CGLIB代理的区别
- 掌握AOP编程方法**：能够独立设计和实现自定义切面
- 应用AOP解决实际问题**：在PKM项目中应用AOP解决日志、缓存、事务等横切关注点
- 培养工程化思维**：学习如何设计可维护、可扩展的横切关注点模块

2.2 实验要求

- 完成基础切面实现**：至少实现日志记录和性能监控两个切面
- 掌握自定义注解**：设计和实现至少一个自定义注解及其切面处理
- 集成到PKM项目**：将AOP功能集成到个人知识管理系统中
- 编写测试用例**：为切面功能编写单元测试
- 提交完整实验报告**：包含设计思路、实现代码、测试结果和分析

3. 实验环境与工具

开发环境

- 操作系统**：Windows 10/11 或 macOS 10.15+
- JDK版本**：JDK 17或以上
- 开发工具**：IntelliJ IDEA 2022.3+ 或 Eclipse 2022+
- 构建工具**：Maven 3.8+ 或 Gradle 7.5+

项目依赖

```
<!-- Spring Boot Starter AOP -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>

<!-- Lombok（可选，简化代码） -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

实验项目结构

```
pkm-web/
├── src/main/java/com/pkm/
│   ├── aspect/                # 切面层
│   │   ├── LoggingAspect.java # 日志记录切面
│   │   ├── PerformanceAspect.java # 性能监控切面
│   │   ├── CacheAspect.java    # 缓存切面
│   │   └── ValidationAspect.java # 参数验证切面
│   ├── annotation/            # 自定义注解
│   │   ├── LogExecutionTime.java # 执行时间日志注解
│   │   ├── OperationLog.java    # 操作日志注解
│   │   └── Cacheable.java       # 缓存注解
│   ├── service/                # 服务层（应用切面）
│   │   ├── NoteService.java
│   │   ├── TagService.java
│   │   └── UserService.java
│   └── config/                  # 配置类
│       └── AopConfig.java
├── src/test/java/com/pkm/aspect/ # 切面测试
└── pom.xml
```

4. 实验内容与步骤

第一部分：基础AOP配置与日志切面实现

实验任务1-1：项目环境配置

1. 创建Spring Boot项目或使用现有PKM项目
2. 添加Spring AOP依赖
3. 创建AOP配置类，启用AOP支持

```
// 任务1-1: 创建AOP配置类
@Configuration
@EnableAspectJAutoProxy(
    proxyTargetClass = true,    // 使用CGLIB代理
    exposeProxy = true        // 暴露代理对象，解决内部调用问题
)
@Slf4j
public class AopConfig {

    @Bean
    @Order(1) // 定义切面执行顺序
    public LoggingAspect loggingAspect() {
        return new LoggingAspect();
    }
}
```

实验任务1-2: 创建日志记录切面

1. 创建日志记录注解
2. 实现日志记录切面
3. 在服务层方法上应用注解

```

// 任务1-2: 创建日志记录注解
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface OperationLog {
    String value() default "";
    OperationType type() default OperationType.QUERY;

    enum OperationType {
        CREATE, UPDATE, DELETE, QUERY, EXPORT
    }
}

// 任务1-2: 实现日志记录切面
@Aspect
@Component
@Slf4j
public class LoggingAspect {

    // 切点: 所有标记@OperationLog的方法
    @Pointcut("@annotation(operationLog)")
    public void operationLogPointcut(OperationLog operationLog) {}

    @Around("operationLogPointcut(operationLog)")
    public Object logOperation(ProceedingJoinPoint joinPoint, OperationLog
operationLog) throws Throwable {
        // 获取方法信息
        MethodSignature signature = (MethodSignature) joinPoint.getSignature();
        String methodName = signature.getMethod().getName();
        String className = joinPoint.getTarget().getClass().getSimpleName();

        // 记录开始日志
        log.info("【操作开始】方法: {}.{}", 操作: {}, 描述: {}",
            className, methodName, operationLog.type(), operationLog.value());

        long startTime = System.currentTimeMillis();

        try {
            // 执行目标方法
            Object result = joinPoint.proceed();
            long endTime = System.currentTimeMillis();

            // 记录成功日志
            log.info("【操作成功】方法: {}.{}", 耗时: {}ms",
                className, methodName, endTime - startTime);

            return result;
        } catch (Exception e) {
            long endTime = System.currentTimeMillis();

```

```

        // 记录失败日志
        log.error("【操作失败】方法: {}.{}", 耗时: {}ms, 异常: {},
            className, methodName, endTime - startTime, e.getMessage(), e);

        throw e;
    }
}

```

实验任务1-3: 应用日志切面

1. 在NoteService的方法上添加@OperationLog注解
2. 测试日志切面效果

```

// 任务1-3: 在服务层应用日志切面
@Service
public class NoteServiceImpl implements NoteService {

    @Override
    @OperationLog(value = "创建新笔记", type = OperationLog.OperationType.CREATE)
    public Note createNote(NoteDTO noteDTO) {
        // 业务逻辑
        log.debug("创建笔记: {}", noteDTO.getTitle());
        return noteRepository.save(convertToEntity(noteDTO));
    }

    @Override
    @OperationLog(value = "根据ID删除笔记", type = OperationLog.OperationType.DELETE)
    public void deleteNote(Long id) {
        log.debug("删除笔记: {}", id);
        noteRepository.deleteById(id);
    }
}

```

实验任务1-4: 编写日志切面测试

```
// 任务1-4: 日志切面测试
@SpringBootTest
@Slf4j
public class LoggingAspectTest {

    @Autowired
    private NoteService noteService;

    @Test
    public void testLoggingAspect() {
        // 测试创建笔记的日志记录
        NoteDTO noteDTO = new NoteDTO();
        noteDTO.setTitle("测试笔记");
        noteDTO.setContent("测试内容");

        Note createdNote = noteService.createNote(noteDTO);
        assertNotNull(createdNote);

        // 验证日志是否记录
        // 可以在控制台查看日志输出，或使用日志收集器验证

        // 测试删除笔记的日志记录
        noteService.deleteNote(createdNote.getId());
    }
}
```

第二部分：性能监控切面与自定义注解

实验任务2-1：创建性能监控切面

1. 创建@LogExecutionTime注解
2. 实现性能监控切面
3. 添加性能阈值配置

```
// 任务2-1: 创建执行时间日志注解
```

```
// 补充代码
```

```
。 。 。
```

```
。 。 。
```

```
。 。 。
```

```
public _____(填空) LogExecutionTime {  
    String value() default "";  
    long threshold() default 1000; // 默认阈值1000ms  
}
```

```
// 任务2-1: 实现性能监控切面
```

```
@Aspect
```

```
@Component
```

```
@Slf4j
```

```
public class PerformanceAspect {
```

```
    // 从配置文件中读取性能阈值
```

```
    @Value("${performance.monitor.threshold:1000}")
```

```
    private long defaultThreshold;
```

```
    // 切点: 所有标记@LogExecutionTime的方法
```

```
    @Pointcut(_____(填空))
```

```
    public void logExecutionTimePointcut(LogExecutionTime logExecutionTime) {}
```

```
    @Around("logExecutionTimePointcut(logExecutionTime)")
```

```
    public Object monitorPerformance(ProceedingJoinPoint joinPoint, LogExecutionTime  
logExecutionTime) throws Throwable {
```

```
        String methodName = joinPoint.getSignature().toShortString();
```

```
        long startTime = System.currentTimeMillis();
```

```
        try {
```

```
            Object result = joinPoint.proceed();
```

```
            long endTime = System.currentTimeMillis();
```

```
            long executionTime = endTime - startTime;
```

```
            // 记录执行时间
```

```
            String logMessage = String.format("方法 %s 执行耗时: %dms", methodName,  
executionTime);
```

```
            // 判断是否超过阈值
```

```
            long threshold = logExecutionTime.threshold() > 0 ?
```

```
logExecutionTime.threshold() : defaultThreshold;
```

```
            if (executionTime > threshold) {
```

```
                log.warn("⚠ 性能警告: {}, 超过阈值 {}ms", logMessage, threshold);
```

```
            } else {
```

```
                log.debug(logMessage);
```

```
            }
```

```
            return result;
```

```

    } catch (Exception e) {
        long endTime = System.currentTimeMillis();
        log.error("✖ 方法 {} 执行异常, 耗时 {}ms, 异常: {}",
            methodName, endTime - startTime, e.getMessage(), e);
        throw e;
    }
}

// 监控所有服务层方法（备用切点）
@Pointcut(_____(填空))
public void serviceLayer() {}

@Around("serviceLayer()")
public Object monitorAllServices(ProceedingJoinPoint joinPoint) throws Throwable {
    // 实现类似的性能监控逻辑
    // 可以添加采样率控制，避免性能开销过大
    return joinPoint.proceed();
}
}

```

实验任务2-2：配置属性文件

```

# application.yml
performance:
  monitor:
    threshold: 500 # 性能监控阈值（ms）
    enabled: true # 是否启用性能监控
    sample-rate: 0.1 # 采样率（仅监控10%的请求）

logging:
  level:
    com.pkm.aspect.PerformanceAspect: DEBUG

```

实验任务2-3：在PKM服务中应用性能监控


```
// 任务2-3: 在关键方法上应用性能监控
@Service
public class SearchServiceImpl implements SearchService {

    @Autowired
    private NoteRepository noteRepository;

    @Override
    @LogExecutionTime(value = "全文搜索", threshold = 2000)
    public List<Note> fullTextSearch(String keyword) {
        // 模拟耗时操作
        simulateHeavySearch(keyword);
        return noteRepository.findByContentContaining(keyword);
    }

    @Override
    @LogExecutionTime(value = "标签搜索")
    public List<Note> searchByTags(List<String> tags) {
        return noteRepository.findByTagsIn(tags);
    }

    private void simulateHeavySearch(String keyword) {
        try {
            // 模拟搜索耗时
            Thread.sleep(800);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

实验任务2-4: 性能监控测试与数据分析

```

// 任务2-4: 性能监控测试
@SpringBootTest
@Slf4j
public class PerformanceAspectTest {

    @Autowired
    private SearchService searchService;

    @Test
    public void testPerformanceMonitoring() {
        // 测试正常性能
        List<Note> results = searchService.searchByTags(Arrays.asList("Java",
"Spring"));
        assertNotNull(results);

        // 测试慢查询（应该触发警告）
        List<Note> slowResults = searchService.fullTextSearch("设计模式");
        assertNotNull(slowResults);

        // 统计测试
        long totalTime = 0;
        int iterations = 10;

        for (int i = 0; i < iterations; i++) {
            long startTime = System.currentTimeMillis();
            searchService.searchByTags(Arrays.asList("测试"));
            long endTime = System.currentTimeMillis();
            totalTime += (endTime - startTime);
        }

        double averageTime = totalTime / (double) iterations;
        log.info("平均执行时间: {}ms", averageTime);
    }
}

```

* 第三部分：缓存切面设计与实现（选做）

实验任务3-1：创建缓存注解

1. 创建@Cacheable注解，支持缓存过期时间
2. 创建@CacheEvict注解，支持缓存清除

```

// 任务3-1: 创建缓存注解
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Cacheable {
    String key(); // 缓存键（支持SpEL表达式）
    long ttl() default 300; // 过期时间（秒）
    TimeUnit unit() default TimeUnit.SECONDS;
}

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface CacheEvict {
    String key(); // 要清除的缓存键
    boolean allEntries() default false; // 是否清除所有缓存
    String pattern() default ""; // 缓存键模式（支持通配符）
}

```

实验任务3-2：实现缓存切面

```

// 任务3-2: 实现缓存切面
@Aspect
@Component
@Slf4j
public class CacheAspect {

    @Autowired
    private CacheManager cacheManager; // Spring Cache抽象

    // 也可以使用Redis等具体实现
    // @Autowired
    // private RedisTemplate<String, Object> redisTemplate;

    @Around("@annotation(cacheable)")
    public Object handleCacheable(ProceedingJoinPoint joinPoint, Cacheable cacheable)
    throws Throwable {
        // 生成缓存键
        String cacheKey = generateCacheKey(joinPoint, cacheable.key());

        // 尝试从缓存获取
        Cache cache = cacheManager.getCache("pkmCache");
        Cache.ValueWrapper cachedValue = cache.get(cacheKey);

        if (cachedValue != null) {
            log.debug("缓存命中: key={}", cacheKey);
            return cachedValue.get();
        }

        log.debug("缓存未命中: key={}", cacheKey);

        // 执行目标方法
        Object result = joinPoint.proceed();

        // 将结果存入缓存
        if (result != null) {
            cache.put(cacheKey, result);
            log.debug("缓存设置成功: key={}, ttl={}{}",
                cacheKey, cacheable.ttl(), cacheable.unit());
        }

        return result;
    }

    @After("@annotation(cacheEvict)")
    public void handleCacheEvict(JoinPoint joinPoint, CacheEvict cacheEvict) {
        Cache cache = cacheManager.getCache("pkmCache");

        if (cacheEvict.allEntries()) {
            // 清除所有缓存
            cache.clear();
        }
    }
}

```

```

        log.debug("清除所有缓存");
    } else if (!cacheEvict.pattern().isEmpty()) {
        // 根据模式清除缓存
        clearCacheByPattern(cacheEvict.pattern());
    } else {
        // 清除指定键的缓存
        String cacheKey = generateCacheKey(joinPoint, cacheEvict.key());
        cache.evict(cacheKey);
        log.debug("清除缓存: key={}", cacheKey);
    }
}

// 生成缓存键（支持SpEL表达式）
private String generateCacheKey(ProceedingJoinPoint joinPoint, String keyPattern)
{
    MethodSignature signature = (MethodSignature) joinPoint.getSignature();
    ExpressionParser parser = new SpelExpressionParser();
    StandardEvaluationContext context = new StandardEvaluationContext();

    // 设置方法参数
    String[] paramNames = signature.getParameterNames();
    Object[] args = joinPoint.getArgs();

    for (int i = 0; i < paramNames.length; i++) {
        context.setVariable(paramNames[i], args[i]);
    }

    // 设置方法信息
    context.setVariable("methodName", signature.getMethod().getName());
    context.setVariable("className",
signature.getDeclaringType().getSimpleName());

    // 解析SpEL表达式
    String evaluatedKey = parser.parseExpression(keyPattern).getValue(context,
String.class);
    return "pkm:" + evaluatedKey;
}

private void clearCacheByPattern(String pattern) {
    // 实现根据模式清除缓存的逻辑
    log.debug("根据模式清除缓存: pattern={}", pattern);
}
}

```

实验任务3-3：配置Spring Cache

```
// 任务3-3: 缓存配置类
@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        // 使用ConcurrentMapCache作为内存缓存
        SimpleCacheManager cacheManager = new SimpleCacheManager();

        List<Cache> caches = new ArrayList<>();
        caches.add(new ConcurrentMapCache("pkmCache"));
        caches.add(new ConcurrentMapCache("userCache"));

        cacheManager.setCaches(caches);
        return cacheManager;

        // 如果要使用Redis, 可以配置RedisCacheManager
        // return RedisCacheManager.builder(redisConnectionFactory).build();
    }
}
```

实验任务3-4: 在PKM中应用缓存

```

// 任务3-4: 应用缓存注解
@Service
public class NoteServiceImpl implements NoteService {

    @Override
    @Cacheable(key = "'note:' + #id")
    public Note getNoteById(Long id) {
        log.debug("从数据库查询笔记: id={}", id);
        return noteRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("笔记不存在"));
    }

    @Override
    @Cacheable(key = "'notes:user:' + #userId")
    public List<Note> getNotesByUser(Long userId) {
        log.debug("从数据库查询用户笔记: userId={}", userId);
        return noteRepository.findById(userId);
    }

    @Override
    @CacheEvict(key = "'note:' + #note.id")
    public Note updateNote(Note note) {
        log.debug("更新笔记: id={}", note.getId());
        Note updatedNote = noteRepository.save(note);

        // 同时清除相关的列表缓存
        Cache cache = cacheManager.getCache("pkmCache");
        cache.evict("notes:user:" + note.getUserId());

        return updatedNote;
    }
}

```

实验任务3-5: 缓存性能测试

```
// 任务3-5: 缓存性能测试
@SpringBootTest
@Slf4j
public class CacheAspectTest {

    @Autowired
    private NoteService noteService;

    @Autowired
    private CacheManager cacheManager;

    @Test
    public void testCachePerformance() {
        // 创建测试数据
        Note note = createTestNote();
        noteService.save(note);

        long startTime, endTime;

        // 第一次查询（应该走数据库）
        startTime = System.currentTimeMillis();
        Note note1 = noteService.getNoteById(note.getId());
        endTime = System.currentTimeMillis();
        long firstQueryTime = endTime - startTime;
        log.info("第一次查询耗时: {}ms (数据库)", firstQueryTime);

        // 第二次查询（应该走缓存）
        startTime = System.currentTimeMillis();
        Note note2 = noteService.getNoteById(note.getId());
        endTime = System.currentTimeMillis();
        long secondQueryTime = endTime - startTime;
        log.info("第二次查询耗时: {}ms (缓存)", secondQueryTime);

        // 验证缓存效果
        assertTrue(secondQueryTime < firstQueryTime, "缓存查询应该更快");

        // 验证缓存命中
        Cache cache = cacheManager.getCache("pkmCache");
        String cacheKey = "note:" + note.getId();
        assertNotNull(cache.get(cacheKey), "缓存中应该存在该数据");

        // 测试缓存清除
        note.setTitle("更新后的标题");
        noteService.updateNote(note);

        // 验证缓存已被清除
        assertNull(cache.get(cacheKey), "更新后缓存应该被清除");
    }
}
```


***第四部分：参数验证切面与综合实践（选做）**

实验任务4-1：创建参数验证切面

1. 创建@ValidateParams注解
2. 实现参数验证切面
3. 集成JSR-303验证框架

```

// 任务4-1: 创建参数验证注解
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface ValidateParams {
    Class<?>[] groups() default {}; // 验证分组
}

// 任务4-1: 实现参数验证切面
@Aspect
@Component
@Slf4j
public class ValidationAspect {

    @Autowired
    private Validator validator; // JSR-303验证器

    @Before("@annotation(validateParams)")
    public void validateParameters(JoinPoint joinPoint, ValidateParams validateParams)
    {
        Object[] args = joinPoint.getArgs();
        MethodSignature signature = (MethodSignature) joinPoint.getSignature();

        for (int i = 0; i < args.length; i++) {
            Object arg = args[i];
            if (arg == null) continue;

            // 检查参数是否有验证注解
            Parameter parameter = signature.getMethod().getParameters()[i];
            if (parameter.isAnnotationPresent(Valid.class)) {
                // 执行JSR-303验证
                Set<ConstraintViolation<Object>> violations =
                    validator.validate(arg, validateParams.groups());

                if (!violations.isEmpty()) {
                    List<String> errorMessages = violations.stream()
                        .map(v -> v.getPropertyPath() + ": " + v.getMessage())
                        .collect(Collectors.toList());

                    throw new ValidationException("参数验证失败: " + String.join(", ",
errorMessages));
                }
            }
        }
    }

    // 可选: 返回值验证
    @AfterReturning(
        pointcut = "@annotation(validateParams)",
        returning = "result"
    )
}

```

```

    )
    public void validateReturnValue(JoinPoint joinPoint, Object result, ValidateParams
validateParams) {
        if (result == null) return;

        Set<ConstraintViolation<Object>> violations =
            validator.validate(result, validateParams.groups());

        if (!violations.isEmpty()) {
            log.warn("方法 {} 返回值验证失败: {}",
                joinPoint.getSignature().getName(), violations);
            // 可以根据需要处理, 如记录日志或抛出异常
        }
    }
}

```

实验任务4-2: 创建验证分组

```

// 任务4-2: 创建验证分组
public interface CreateGroup {}
public interface UpdateGroup {}

// 任务4-2: 应用验证的DTO
@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class NoteDTO {

    @Null(groups = CreateGroup.class, message = "创建时ID必须为空")
    @NotNull(groups = UpdateGroup.class, message = "更新时ID不能为空")
    private Long id;

    @NotBlank(message = "标题不能为空")
    @Size(min = 1, max = 100, message = "标题长度必须在1-100之间")
    private String title;

    @NotBlank(message = "内容不能为空")
    @Size(min = 1, max = 10000, message = "内容长度必须在1-10000之间")
    private String content;

    @Size(max = 10, message = "标签最多10个")
    private List<String> tags;

    @NotNull(message = "用户ID不能为空")
    private Long userId;
}

```

实验任务4-3：在服务层应用参数验证

```
// 任务4-3：应用参数验证
@Service
@Validated
public class NoteServiceImpl implements NoteService {

    @Override
    @ValidateParams(groups = CreateGroup.class)
    public Note createNote(@Valid NoteDTO noteDTO) {
        log.debug("创建笔记: {}", noteDTO.getTitle());
        return noteRepository.save(convertToEntity(noteDTO));
    }

    @Override
    @ValidateParams(groups = UpdateGroup.class)
    public Note updateNote(@Valid NoteDTO noteDTO) {
        log.debug("更新笔记: id={}", noteDTO.getId());

        Note existingNote = noteRepository.findById(noteDTO.getId())
            .orElseThrow(() -> new ResourceNotFoundException("笔记不存在"));

        updateNoteFromDTO(existingNote, noteDTO);
        return noteRepository.save(existingNote);
    }
}
```

实验任务4-4：综合实践：PKM系统AOP集成

1. 创建统一的AOP配置
2. 配置切面执行顺序
3. 创建AOP测试控制器

```

// 任务4-4: 统一的AOP配置
@Configuration
@EnableAspectJAutoProxy
public class GlobalAopConfig {

    // 定义切面执行顺序常量
    public static final int ORDER_VALIDATION = 100;    // 验证最先执行
    public static final int ORDER_CACHE = 200;        // 缓存
    public static final int ORDER_PERFORMANCE = 300;   // 性能监控
    public static final int ORDER_LOGGING = 400;       // 日志记录

    @Bean
    @Order(ORDER_VALIDATION)
    public ValidationAspect validationAspect() {
        return new ValidationAspect();
    }

    @Bean
    @Order(ORDER_CACHE)
    public CacheAspect cacheAspect() {
        return new CacheAspect();
    }

    @Bean
    @Order(ORDER_PERFORMANCE)
    public PerformanceAspect performanceAspect() {
        return new PerformanceAspect();
    }

    @Bean
    @Order(ORDER_LOGGING)
    public LoggingAspect loggingAspect() {
        return new LoggingAspect();
    }
}

// 任务4-4: AOP测试控制器
@RestController
@RequestMapping("/api/test/aop")
@Slf4j
public class AopTestController {

    @Autowired
    private NoteService noteService;

    @PostMapping("/create")
    public ResponseEntity<Note> createNoteWithAOP(@RequestBody NoteDTO noteDTO) {
        log.info("测试AOP集成 - 创建笔记");
        Note note = noteService.createNote(noteDTO);
        return ResponseEntity.ok(note);
    }
}

```

```

    }

    @GetMapping("/{id}")
    public ResponseEntity<Note> getNoteWithAOP(@PathVariable Long id) {
        log.info("测试AOP集成 - 获取笔记");
        Note note = noteService.getNoteById(id);
        return ResponseEntity.ok(note);
    }

    @GetMapping("/performance")
    public ResponseEntity<String> testPerformance() {
        log.info("测试AOP集成 - 性能监控");

        long startTime = System.currentTimeMillis();

        // 模拟多次查询，测试性能监控
        for (int i = 0; i < 5; i++) {
            noteService.getNotesByUser(1L);
        }

        long endTime = System.currentTimeMillis();
        String result = String.format("测试完成，总耗时：%dms", endTime - startTime);

        return ResponseEntity.ok(result);
    }
}

```

实验任务4-5：编写综合测试用例

```

// 任务4-5: 综合测试用例
@SpringBootTest
@AutoConfigureMockMvc
@Slf4j
public class AopIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private NoteService noteService;

    @Test
    public void testAopIntegration() throws Exception {
        // 1. 测试参数验证切面
        NoteDTO invalidNote = NoteDTO.builder()
            .title("") // 空标题, 应该触发验证异常
            .content("测试内容")
            .userId(1L)
            .build();

        mockMvc.perform(post("/api/test/aop/create")
            .contentType(MediaType.APPLICATION_JSON)
            .content(asJsonString(invalidNote))
            .andExpect(status().isBadRequest()); // 应该返回400错误

        // 2. 测试日志和性能监控切面
        NoteDTO validNote = NoteDTO.builder()
            .title("AOP测试笔记")
            .content("测试AOP功能集成")
            .userId(1L)
            .build();

        String response = mockMvc.perform(post("/api/test/aop/create")
            .contentType(MediaType.APPLICATION_JSON)
            .content(asJsonString(validNote))
            .andExpect(status().isOk())
            .andReturn()
            .getResponse()
            .getContentAsString());

        Note createdNote = objectMapper.readValue(response, Note.class);

        // 3. 测试缓存切面
        // 第一次查询
        long startTime = System.currentTimeMillis();
        noteService.getNoteById(createdNote.getId());
        long firstQueryTime = System.currentTimeMillis() - startTime;

        // 第二次查询 (应该命中缓存)
    }
}

```

```
        startTime = System.currentTimeMillis();
        noteService.getNoteById(createdNote.getId());
        long secondQueryTime = System.currentTimeMillis() - startTime;

        assertTrue(secondQueryTime < firstQueryTime, "缓存应该生效");

        // 4. 测试性能监控
        mockMvc.perform(get("/api/test/aop/performance"))
                .andExpect(status().isOk());
    }

    @Test
    public void testAopExecutionOrder() {
        // 验证切面执行顺序
        // 通过日志分析或使用专门的跟踪切面
        log.info("测试切面执行顺序...");

        // 创建一个跟踪切面来验证执行顺序
        // 实现略...
    }

    private String asJsonString(Object obj) throws JsonProcessingException {
        ObjectMapper mapper = new ObjectMapper();
        return mapper.writeValueAsString(obj);
    }
}
```

常见问题与解决方案

问题1: AOP切面不生效

可能原因:

1. 类没有被Spring管理（缺少@Component等注解）
2. 切点表达式写错
3. 方法被内部调用（非代理调用）

解决方案:


```

// 1. 确保切面类被Spring管理
@Aspect
@Component // 必须有这个注解
public class MyAspect { ... }

// 2. 使用正确的切点表达式
@Pointcut("execution(* com.pkm.service.*(..))") // 检查包路径

// 3. 解决内部调用问题
@Service
public class MyService {
    @Autowired
    private ApplicationContext context;

    public void outerMethod() {
        // 错误：直接调用，AOP不生效
        innerMethod();

        // 正确：通过代理调用
        MyService proxy = context.getBean(MyService.class);
        proxy.innerMethod();
    }

    @LogExecutionTime
    public void innerMethod() { ... }
}

```

问题2：循环依赖

可能原因：切面依赖了被代理的服务，而服务又依赖了切面

解决方案：

```

// 1. 使用@Lazy延迟加载
@Component
public class MyAspect {
    @Lazy // 延迟加载，打破循环依赖
    @Autowired
    private MyService myService;
}

// 2. 重构代码，避免循环依赖

```

问题3：性能问题

可能原因：切面逻辑过于复杂或切点匹配过多方法

解决方案：

1. 优化切点表达式，精确匹配
 2. 简化切面逻辑
 3. 使用条件判断减少不必要的处理
 4. 考虑使用编译时织入（AspectJ）
-

4. 实验提交方式

1. **代码提交**：git提交后设置标签 week13_aop，推送到服务器
 2. 提交截止时间：本周日晚
-

5. 实验注意事项

1. **代码规范**：遵循Java编码规范，添加必要注释
 2. **版本控制**：使用Git进行版本管理，提交记录清晰
 3. **测试驱动**：先写测试用例，再实现功能
 4. **性能考虑**：注意AOP对系统性能的影响
 5. **安全注意**：验证切面要注意防止SQL注入等安全问题
 6. **文档完整**：确保README和API文档完整
-

实验资源推荐

在线资源

1. [使用AOP - Java教程 - 廖雪峰的官方网站](#)